

```

# =====
# EXPERIMENT: NOISE REMOVAL COMPARISON
# Comparative Study of Different Noise Removal Techniques
# Google Colab - OpenCV Python
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt

# -----
# STEP 1: Read the uploaded image
# -----

image_path = "/content/image1.jpg" # Change only the filename if needed

img = cv2.imread(image_path)

if img is None:
    print("Image not found. Check the image name in /content/")
else:

    # Convert to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # -----
    # STEP 2: Add Gaussian Noise
    # -----

    mean = 0
    sigma = 25

    noise = np.random.normal(
        mean,
        sigma,
        gray.shape
    )

    noisy_image = gray.astype(np.float32) + noise

    # Keep pixel values between 0 and 255
    noisy_image = np.clip(noisy_image, 0, 255).astype(np.uint8)

    # -----
    # STEP 3: Apply Different Noise Removal Filters
    # -----

    # 1. Mean Filter
    mean_filter = cv2.blur(noisy_image, (5, 5))

    # 2. Median Filter
    median_filter = cv2.medianBlur(noisy_image, 5)

    # 3. Gaussian Filter
    gaussian_filter = cv2.GaussianBlur(
        noisy_image,
        (5, 5),
        1.5
    )

```

```

# 4. Bilateral Filter
bilateral_filter = cv2.bilateralFilter(
    noisy_image,
    9,
    75,
    75
)

# 5. Non-Local Means Filter
nlm_filter = cv2.fastNlMeansDenoising(
    noisy_image,
    None,
    10,
    7,
    21
)

# -----
# STEP 4: Display Original, Noisy and Filtered Images
# -----

images = [
    gray,
    noisy_image,
    mean_filter,
    median_filter,
    gaussian_filter,
    bilateral_filter,
    nlm_filter
]

titles = [
    "Original Image",
    "Noisy Image (Gaussian)",
    "Mean Filter (5x5)",
    "Median Filter (5x5)",
    "Gaussian Filter (5x5)",
    "Bilateral Filter",
    "NLM Filter"
]

plt.figure(figsize=(16, 10))

for i in range(len(images)):
    plt.subplot(2, 4, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

# -----
# STEP 5: Save All Outputs
# -----

cv2.imwrite("/content/noisy_image.jpg", noisy_image)
cv2.imwrite("/content/mean_filter.jpg", mean_filter)
cv2.imwrite("/content/median_filter.jpg", median_filter)
cv2.imwrite("/content/gaussian_filter.jpg", gaussian_filter)

```

```

cv2.imwrite( /content/bilateral_filter.jpg , bilateral_filter)
cv2.imwrite("/content/nlm_filter.jpg", nlm_filter)

print("All filtered images saved successfully!")

# -----
# STEP 6: Analysis
# -----

print("\n===== NOISE REMOVAL ANALYSIS =====")

print("\n1. Mean Filter:")
print(" - Reduces noise effectively.")
print(" - Produces noticeable blurring.")
print(" - Fine image details may be lost.")

print("\n2. Median Filter:")
print(" - Very effective for salt-and-pepper noise.")
print(" - Preserves edges better than the mean filter.")
print(" - Reduces noise while maintaining details.")

print("\n3. Gaussian Filter:")
print(" - Effective for Gaussian noise.")
print(" - Produces smooth results.")
print(" - May slightly blur edges.")

print("\n4. Bilateral Filter:")
print(" - Removes noise while preserving edges.")
print(" - Maintains important image details.")
print(" - Computationally more expensive.")

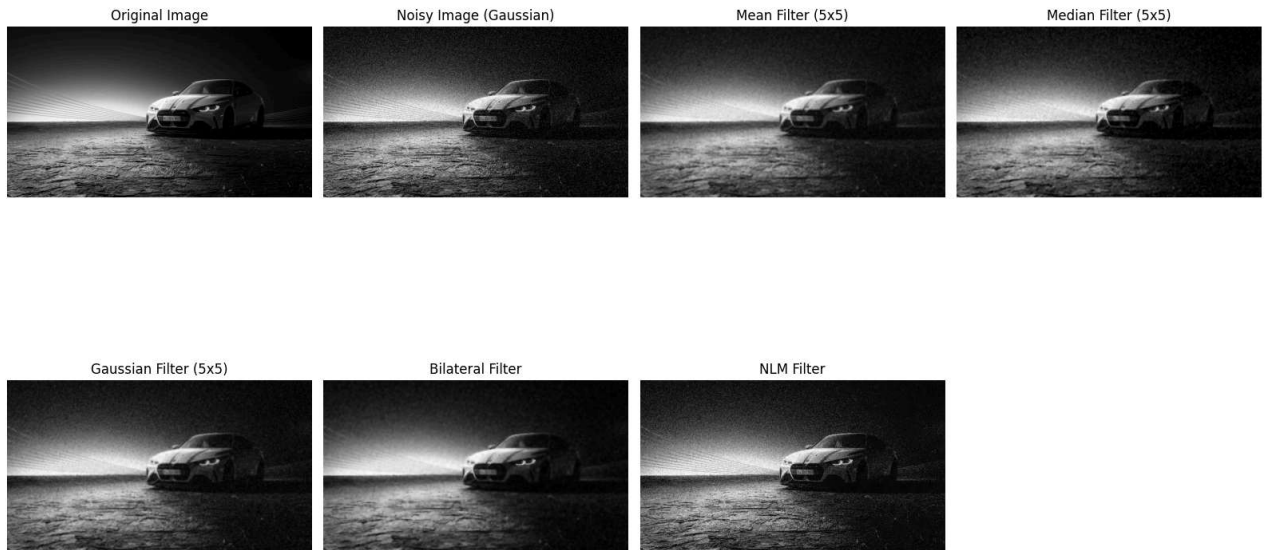
print("\n5. Non-Local Means (NLM):")
print(" - Provides strong noise reduction.")
print(" - Preserves image details very well.")
print(" - Computationally more expensive than simple filters.")

# -----
# STEP 7: Conclusion
# -----

print("\n===== CONCLUSION =====")

print("Different filters have different noise removal capabilities.")
print("Mean filtering reduces noise but causes more blurring.")
print("Median filtering is effective for salt-and-pepper noise.")
print("Gaussian filtering is suitable for Gaussian noise.")
print("Bilateral filtering preserves edges while reducing noise.")
print("NLM provides strong noise reduction with good detail preservation.")

```



All filtered images saved successfully!

===== NOISE REMOVAL ANALYSIS =====

1. Mean Filter:
 - Reduces noise effectively.
 - Produces noticeable blurring.
 - Fine image details may be lost.
2. Median Filter:
 - Very effective for salt-and-pepper noise.
 - Preserves edges better than the mean filter.
 - Reduces noise while maintaining details.
3. Gaussian Filter:
 - Effective for Gaussian noise.
 - Produces smooth results.
 - May slightly blur edges.
4. Bilateral Filter:
 - Removes noise while preserving edges.
 - Maintains important image details.
 - Computationally more expensive.
5. Non-Local Means (NLM):
 - Provides strong noise reduction.
 - Preserves image details very well.
 - Computationally more expensive than simple filters.

===== CONCLUSION =====

Different filters have different noise removal capabilities.
 Mean filtering reduces noise but causes more blurring.
 Median filtering is effective for salt-and-pepper noise.
 Gaussian filtering is suitable for Gaussian noise.
 Bilateral filtering preserves edges while reducing noise.
 NLM provides strong noise reduction with good detail preservation.

